SRTF Shortest Remaining Time First CPU Scheduling

---

1. AIM

To simulate **SRTF CPU Scheduling** using structures and compute:

- Completion Time
- Turnaround Time
- Waiting Time
- Average Waiting Time

for a set of processes with given arrival and burst times.

---

2. THEORY

Shortest Remaining Time First (SRTF)

- It is the **preemptive version** of SJF.
- At every time unit, the scheduler picks the process with the **shortest remaining time**.
- If a new process arrives with shorter remaining burst time → **preemption happens**.
- Produces optimal waiting time but is difficult to implement.

---

3. ALGORITHM

1. Input **n** processes with arrival time and burst time.
2. Create structure array storing each process's information.
3. At each time unit:
    - Select process with smallest **remaining time** that has arrived.
    - Decrement remaining time.
    - If remaining time = 0 → mark completion.
4. After all processes finish:
    - Compute Turnaround Time
    - Compute Waiting Time
5. Compute Average Waiting Time.

---

4. C PROGRAM – SRTF Using Structures

```
#include <stdio.h>

int main() {
    int n, i, j;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int bt[n], at[n], rt[n], wt[n], tat[n], ct[n];
    int completed = 0, t = 0, min_bt, shortest;
    float total_wt = 0, total_tat = 0;
    int is_completed[n];

    // Initialize
    for(i = 0; i < n; i++) is_completed[i] = 0;

    // Input Arrival and Burst Times
    for(i = 0; i < n; i++) {
        printf("Enter arrival time and burst time for process P%d: ", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i]; // Remaining time initially = burst time
    }

    printf("\nGantt Chart (process execution order):\n");

    while(completed != n) {
        min_bt = 100000; // a large number
        shortest = -1;

        // Find process with shortest remaining time at time t
```

```
    for(i = 0; i < n; i++) {
        if(at[i] <= t && !is_completed[i] && rt[i] < min_bt && rt[i] > 0) {
            min_bt = rt[i];
            shortest = i;
        }
    }

    if(shortest == -1) {
        t++; // No process available, idle CPU
        continue;
    }

    // Execute process for 1 unit
    rt[shortest]--;
    printf("P%d ", shortest+1);
    t++;

    // If process finished
    if(rt[shortest] == 0) {
        completed++;
        is_completed[shortest] = 1;
        ct[shortest] = t; // Completion time
        tat[shortest] = ct[shortest] - at[shortest]; // Turnaround time
        wt[shortest] = tat[shortest] - bt[shortest]; // Waiting time
        total_wt += wt[shortest];
        total_tat += tat[shortest];
    }
}

// Print results
printf("\n\nProcess\tAT\tBT\tCT\tWT\tTAT\n");
```

```
    for(i = 0; i < n; i++) {
        printf("P%d\t%d\t%d\t%d\t%d\t%d\n", i+1, at[i], bt[i], ct[i], wt[i], tat[i]);
    }

    printf("\nAverage Waiting Time = %.2f", total_wt/n);
    printf("\nAverage Turnaround Time = %.2f\n", total_tat/n);

    return 0;
}
```

Sample Input

Enter number of processes: 4

Enter arrival time and burst time for process P1: 0 8

Enter arrival time and burst time for process P2: 1 5

Enter arrival time and burst time for process P3: 2 3

Enter arrival time and burst time for process P4: 3 2

Gantt Chart (process execution order):

P1 P2 P3 P3 P3 P4 P4 P2 P2 P2 P2 P1 P1 P1 P1 P1 P1 P1

Process	AT	BT	CT	WT	TAT
P1	0	8	18	10	18
P2	1	5	11	5	10
P3	2	3	5	0	3
P4	3	2	7	2	4

Average Waiting Time = 4.25

Average Turnaround Time = 8.75